

CS224N: Natural Language Processing with Deep Learning

Lecture 1: Introduction and Word Vectors

Stanford University – Spring 2024

Instructor: Christopher Manning

Contents

1	Human Language and the Power of Language	3
1.1	Language as Cognitive Scaffolding	3
1.2	Writing: A Revolutionary Cognitive Tool	3
1.3	Understanding Language Beyond Words	3
2	Word Representations in NLP	3
2.1	Traditional Approaches	3
2.1.1	WordNet: A Thesaurus-Based Approach	3
2.1.2	One-Hot Vectors: Discrete Symbol Representation	4
2.2	Distributional Semantics	4
2.3	Word Vectors (Embeddings)	4
3	Word2Vec: Learning Word Vectors	5
3.1	Overview	5
3.2	Terminology	5
3.3	Skip-Gram Model	5
3.4	Objective Function	6
3.4.1	Likelihood Function	6
3.4.2	Objective Function (Cost/Loss)	6
3.5	Probability Calculation	6
3.5.1	Two Vectors per Word	6
3.5.2	Softmax Function	6
3.6	Model Parameters	7
4	Training: Gradient Descent	7
4.1	Computing Gradients	7
4.1.1	Gradient with Respect to Center Vector	7
4.1.2	Gradient with Respect to Context Vector	7
4.2	Optimization Algorithms	8
4.2.1	Gradient Descent	8
4.2.2	Stochastic Gradient Descent (SGD)	8
5	Properties and Applications	8
5.1	Semantic Spaces	8
5.2	Current Embeddings: Context-Dependent	9
5.3	From Word2Vec to Modern NLP	9
6	Key Takeaways	9

7	Course Information	9
7.1	Logistics	9
7.2	Grading	9
7.3	Learning Goals	10

1 Human Language and the Power of Language

1.1 Language as Cognitive Scaffolding

Language has been fundamental to human advancement in several key ways:

Key Insight

Language enables higher-level cognition: Most higher-level cognitive processes involve thinking in language. When we reflect on complex situations (e.g., “My friend seemed upset about what I said last night, I should probably work out how to fix that”), we think and plan using language as a scaffolding structure.

1.2 Writing: A Revolutionary Cognitive Tool

- **Timeline:** Writing is remarkably recent – approximately 5,000 years old
- **Impact:** Writing proved to be an extraordinary cognitive tool that gave humanity an enormous advantage
- **Key capability:** Writing enables sharing knowledge across *time and space*, not just with people within immediate physical proximity (within 50 feet)
- **Result:** Thanks to writing, we now have advanced technologies that developed in a very short period of time

1.3 Understanding Language Beyond Words

“The common misconception is that language use has primarily to do with words and what they mean. It doesn’t. It has primarily to do with people and what they mean.”

Key distinctions:

- Language is *not* simply words – words change with every generation (e.g., Gen Z language)
- Language can be extended and evolves through usage
- **Humans:** Take steps and actions by involving language
- **AI:** Operates with numbers, not language in the human sense

2 Word Representations in NLP

2.1 Traditional Approaches

2.1.1 WordNet: A Thesaurus-Based Approach

WordNet is a lexical database that represents word relations:

- Contains **synonyms** and **hypernyms** (“is-a” relationships)
- Example hierarchy: Panda → Carnivore → Placental → Mammal → ... → Entity

Limitations of WordNet:

1. **Misses nuance:** Lists “proficient” as synonym of “good”, which is only correct in some contexts

2. **Cannot keep up-to-date:** Missing new meanings like “wicked”, “badass”, “nifty”, “wizard”, “genius”, “ninja”, “bombest”
3. **Subjective:** Requires human labor to create and adapt
4. **Poor for computational meaning:** Not suitable for computing accurate word similarity

2.1.2 One-Hot Vectors: Discrete Symbol Representation

Traditional NLP represents words as discrete symbols using **one-hot encoding**:

$$\begin{aligned}\text{motel} &= [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0] \\ \text{hotel} &= [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0]\end{aligned}$$

Problems with one-hot vectors:

- Vector dimension = vocabulary size (e.g., 500,000+)
- All vectors are **orthogonal** to each other
- **No natural notion of similarity** between words
- Example: “motel” and “hotel” have zero similarity despite being semantically related

2.2 Distributional Semantics

Definition 1 (Distributional Semantics). *A word’s meaning is given by the words that frequently appear close-by in text.*

“You shall know a word by the company it keeps” – J.R. Firth (1957)

Core idea:

- When word w appears in text, its **context** is the set of words appearing nearby (within a fixed-size window)
- Use many contexts of w to build up a representation of w

Example contexts for “banking”:

- “...government debt problems turning into **banking** crises as happened in 2009...”
- “...saying that Europe needs unified **banking** regulation to replace the hodgepodge...”
- “...India has just given its **banking** system a shot in the arm...”

2.3 Word Vectors (Embeddings)

Definition 2 (Word Vector). *A dense vector representation for each word, chosen so that it is similar to vectors of words appearing in similar contexts.*

Properties:

- Also called **word embeddings** or **neural word representations**
- Form a **distributed representation**

- Similarity measured via **dot product** (scalar product)

Example:

$$\text{banking} = \begin{bmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{bmatrix}, \quad \text{monetary} = \begin{bmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{bmatrix}$$

Key insight: Words with similar meanings have similar vectors, creating semantic spaces where related concepts cluster together.

3 Word2Vec: Learning Word Vectors

3.1 Overview

Word2Vec (Mikolov et al., 2013) is a framework for learning word vectors from large text corpora.

Word2Vec Core Idea

1. Start with a large corpus of text
2. Represent every word in vocabulary by a vector
3. Go through each position t in text with:
 - Center word c
 - Context (“outside”) words o
4. Use similarity of word vectors for c and o to calculate $P(o|c)$
5. Adjust word vectors to maximize this probability

3.2 Terminology

- **Corpus:** Latin for “body”, plural is “corpora” – a large collection of text
- **Word type:** The word “problems” wherever it occurs (unique word)
- **Word token:** One specific instance of the word “problems” in text

3.3 Skip-Gram Model

The Skip-Gram model predicts context words given a center word.

Example: For the sentence fragment:

... problems turning into **banking** crises as ...

With center word “banking” and window size $m = 2$:

- Context words: “problems”, “turning”, “into”, “crises”
- Compute: $P(w_{t-2}|w_t)$, $P(w_{t-1}|w_t)$, $P(w_{t+1}|w_t)$, $P(w_{t+2}|w_t)$

3.4 Objective Function

3.4.1 Likelihood Function

For each position $t = 1, \dots, T$, predict context words within window size m :

$$L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j}|w_t; \theta) \quad (1)$$

where θ represents all model parameters.

3.4.2 Objective Function (Cost/Loss)

The objective is the **average negative log likelihood**:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j}|w_t; \theta) \quad (2)$$

Why use negative log likelihood?

1. **Products $\rightarrow$ Sums:** Log converts products to sums (easier to compute)
2. **Minimization convention:** Algorithms use gradient descent (minimization)
3. **Normalization:** Averaging by T prevents objective from growing with corpus size

Goal: Minimizing $J(\theta) \Leftrightarrow$ Maximizing predictive accuracy

3.5 Probability Calculation

3.5.1 Two Vectors per Word

Each word w has two vector representations:

- $\mathbf{v}_w$: when w is a **center word**
- $\mathbf{u}_w$: when w is a **context word**

3.5.2 Softmax Function

For center word c and context word o :

$$P(o|c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \quad (3)$$

Components:

1. **Dot product $\mathbf{u}_o^\top \mathbf{v}_c$:** Measures similarity between o and c
 - Larger dot product $\rightarrow$ higher probability
 - $\mathbf{u}^\top \mathbf{v} = \sum_{i=1}^d u_i v_i$
2. **Exponentiation:** Makes all values positive
3. **Normalization:** Denominator ensures probabilities sum to 1

Softmax Function

General form: $\mathbb{R}^n \rightarrow (0, 1)^n$

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)} = p_i \quad (4)$$

- “max”: amplifies probability of largest x_i
- “soft”: still assigns some probability to smaller x_i

3.6 Model Parameters

Total parameters: With d -dimensional vectors and vocabulary size V :

- Each word has 2 vectors (center and context)
- Total parameters: $2 \times V \times d$
- Example: $V = 400,000$, $d = 100 \rightarrow 80$ million parameters

These word vectors are the *only* parameters in the model.

The miracle of deep learning: Starting with random word vectors and a pile of text, through mathematical optimization, we obtain meaningful representations that capture semantic similarity!

4 Training: Gradient Descent

4.1 Computing Gradients

To train, we compute gradients of the objective function with respect to all parameters and update them iteratively.

4.1.1 Gradient with Respect to Center Vector

The partial derivative with respect to the center word vector $\mathbf{v}_c$:

$$\frac{\partial}{\partial \mathbf{v}_c} \log P(o|c) = \mathbf{u}_o - \sum_{w \in V} P(w|c) \mathbf{u}_w \quad (5)$$

Interpretation:

- $\mathbf{u}_o$: The actual observed context word vector
- $\sum_{w \in V} P(w|c) \mathbf{u}_w$: Weighted average (expectation) of all context vectors based on current model
- At optimum: observed = expected $\rightarrow$ derivative = 0

4.1.2 Gradient with Respect to Context Vector

Similarly, we can compute $\frac{\partial J}{\partial \mathbf{u}_o}$ (outside vector parameters).

4.2 Optimization Algorithms

4.2.1 Gradient Descent

Algorithm:

1. Initialize parameters θ (randomly)
2. Compute gradient: $\nabla_{\theta} J(\theta)$
3. Update: $\theta^{\text{new}} = \theta^{\text{old}} - \alpha \nabla_{\theta} J(\theta)$
4. Repeat until convergence

where α is the **learning rate** (step size).

Update for single parameter:

$$\theta_j^{\text{new}} = \theta_j^{\text{old}} - \alpha \frac{\partial J(\theta)}{\partial \theta_j} \quad (6)$$

4.2.2 Stochastic Gradient Descent (SGD)

Problem with standard gradient descent:

- $J(\theta)$ is a function of *all* windows in corpus (potentially billions)
- Very expensive to compute full gradient
- Would wait very long before making a single update

Solution: Stochastic Gradient Descent

- Repeatedly sample windows
- Update parameters after each window (or small batch)
- Much faster convergence in practice

Variant: Mini-Batch Gradient Descent

- Compute gradient over small batch of examples
- Balance between efficiency and stability

5 Properties and Applications

5.1 Semantic Spaces

Word embeddings create **semantic spaces** where:

- Words with similar meanings are close together
- Semantic relationships are preserved
- Analogies can be captured through vector arithmetic

5.2 Current Embeddings: Context-Dependent

Limitation of Word2Vec:

- Each word has a single embedding
- Embedding is an average over all senses
- Example: “bank” could mean river bank or financial institution

Modern solution: Contextual embeddings (e.g., BERT, GPT) that generate different vectors based on context.

5.3 From Word2Vec to Modern NLP

Historical progression:

- **GPT-2 (2019):** First model showing strong LLM capabilities
 - Produces grammatically correct sentences
 - Shows understanding (e.g., knows theft is stealing)
- **Modern models:** Multimodal, operate across text, images, etc.
- **Current state:** ChatGPT, GPT-4, and foundation models

6 Key Takeaways

Main Learning Objectives

1. **Word meaning can be represented as high-dimensional vectors of real numbers**
2. **Distributional semantics:** A word’s meaning comes from its contexts
3. **Word2Vec:** Framework for learning vectors by predicting context words
4. **Optimization:** Use (stochastic) gradient descent to learn parameters
5. **Foundation:** These concepts underlie modern NLP and large language models

7 Course Information

7.1 Logistics

- **Instructor:** Christopher Manning
- **Time:** Tu/Th 4:30–5:50 PM Pacific (Nvidia Auditorium)
- **Website:** <http://cs224n.stanford.edu/>

7.2 Grading

- Assignments (4): 48% (6% + 3×14%)
- Final Project: 49% (proposal 8%, milestone 6%, poster 3%, report 32%)
- Participation: 3%
- Late policy: 6 free late days; 1% off per day after

7.3 Learning Goals

1. Foundations of deep learning for NLP (word vectors, RNNs, attention, transformers)
2. Understanding of human language and computational challenges
3. Ability to build NLP systems in PyTorch